

PRD Review and Implementation Guidance

This document presents a structured review of the PRD and highlights the technical considerations needed to turn it into a scalable SaaS product.

1. What the product is

This is not a small CRUD-style application. It is a broker-connected trading journal with analytics and an insight engine. The product can scale to 10k+ users, but only if the architecture and sync strategy are designed correctly from the beginning.

2. Core layers of the system

User and authentication layer

- Signup and login
- Subscription handling through Stripe
- User isolation, which is essential for a multi-tenant SaaS platform

Broker integration layer

- Support for MT4 and MT5 through a bridge such as MetaApi
- Support for Binance and other crypto exchanges through direct APIs
- This is not a normal API integration; it is a continuous sync system

Data sync engine

- Pull trades every 5–15 minutes
- Normalize all data into a single trades table
- Deduplicate using external_trade_id
- Store clean and unified trade data

Analytics engine

- Session analysis for London, New York, and Asian sessions
- Symbol performance tracking
- Hold time bucket analysis
- Win/loss pattern analysis

- R-multiple calculations

Insight engine

- Converts data into plain-language insights
- Example: "Your London session win rate is 71%"
- This can run nightly or be computed on demand and cached

3. Can it scale to 10K+ users?

Yes, but only if the system is designed around asynchronous processing from the beginning.

Required architecture choices

- Use a worker queue system instead of synchronous request-to-compute workflows
- Use Laravel Queue with Redis or RabbitMQ for better scalability
- Create separate workers for trade sync, normalization, analytics, and insight computation

Database design

The PRD is already strong in its use of a single trades table, normalized schema, and row-level security concepts for multi-user isolation. For larger scale, add indexes for `user_id`, `closed_at`, and `symbol`, and consider partitioning later if needed.

Broker sync bottleneck

Broker synchronization is the major bottleneck because each user may require external polling or a bridge connection. If all users sync at the same interval, the load can become very heavy. The solution is staggered sync scheduling and priority-based syncing.

A practical example is to sync active users every 5 minutes and inactive users every 30–60 minutes.

Insight engine and caching

Insights should not be calculated on each page load. Instead, they should be precomputed and stored in an `insights_cache` table so the home screen can retrieve results instantly.

Redis usage

- Job queue handling
- Insight caching
- Rate limiting for API calls
- Sync locks to avoid duplicate processing

4. Recommended implementation approach

Laravel is the better choice for this product because it supports queues, cron jobs, workers, caching, job retries, and multi-provider integrations. CodeIgniter would be less suitable for this scale-heavy SaaS architecture.

Suggested stack

- Backend: Laravel 11
- Authentication: Sanctum or Passport
- Queue system: Redis with Laravel Horizon
- Database: PostgreSQL recommended; MySQL is acceptable but less ideal for analytics-heavy workloads

5. Risks to manage

- **MT4/MT5 dependency risk:** bridge provider costs can change and affect margins.
- **Insight engine complexity risk:** overbuilding machine learning too early can slow product development.
- **Sync overload risk:** without a queue system, the platform may become unstable at 2k–3k users.

6. MVP roadmap

- **Phase 1:** authentication, broker connection, trade sync, trades table, and basic UI
- **Phase 2:** queue system, scheduled sync, and trade normalization
- **Phase 3:** analytics engine and session/symbol statistics
- **Phase 4:** insight engine and home-screen summaries
- **Phase 5:** Stripe billing, production scaling, and optimization

7. Hosting guidance

Hostinger can be suitable for an MVP and early users, but it is not the best long-term platform for a broker-connected SaaS with background sync workers and heavier analytics workloads. A hybrid approach is often the most practical path: start small, then move backend workers and database services to a more scalable platform as usage grows.

8. Final takeaway

This is a real SaaS fintech analytics product rather than a small project. The product concept is strong, but its long-term success depends heavily on queue-based sync design, Redis-based

caching, precomputed insights, and a robust background-job architecture.